

Motor Communication Console 操作手冊

Motor Communication Console 是一套專注於「電腦與馬達通訊」的桌面工具。它可以透過 STM32、Direct CAN 或模擬後端送出馬達命令，並顯示馬達回授與 CAN 通訊狀態。

本版本不包含額外量測裝置、資料記錄或離線分析功能。

1. 安全須知

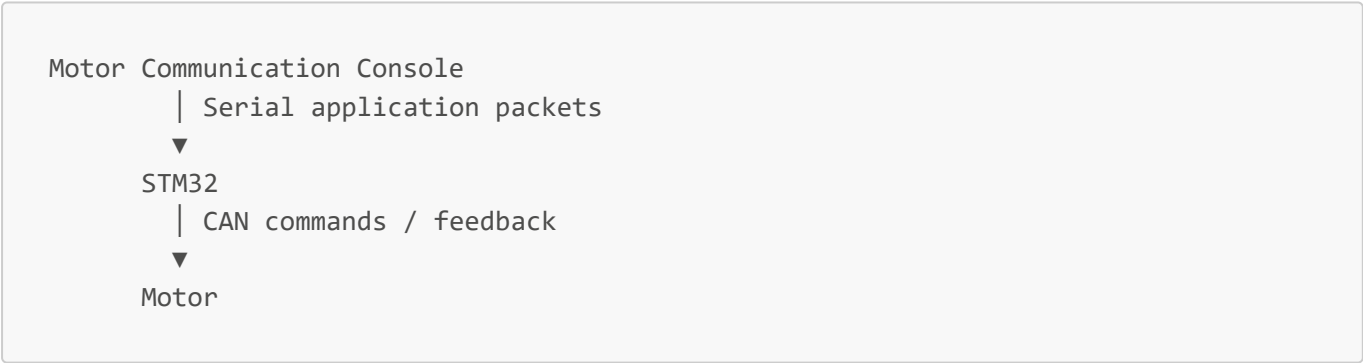
在接上實體馬達前，請先閱讀並遵守以下規則：

- 1. 必須準備可獨立切斷馬達電源或驅動器輸出的 Hardware E-Stop。
- 2. **Software Stop / Motor Idle** 只是透過目前的通訊路徑送出 Idle 命令，不能取代 Hardware E-Stop。
- 3. 第一次測試時應架空馬達或卸除負載，並清空旋轉範圍。
- 4. 從低 torque、current、velocity 與 Kp/Kd 開始測試。
- 5. 操作人員應能在不靠近馬達活動範圍的情況下觸發 Hardware E-Stop。
- 6. 若回授方向、尺度、溫度、電流或 CAN 狀態異常，立即停止操作並切斷硬體輸出。

軟體會檢查扭矩命令、馬達回報電流、溫度、錯誤碼與回授時效，但任何軟體保護都可能因通訊中斷、韌體錯誤或硬體故障而失效。

2. 系統架構

預設的實機通訊路徑如下：



職責劃分：

- Python GUI：使用者設定、邏輯命令、狀態顯示與安全檢查。
- STM32：封包處理、固定頻率 CAN 控制、馬達回授解析、CAN 統計及 watchdog。
- Motor：執行控制命令並透過 CAN 回傳位置、速度、電流、溫度與錯誤碼。

3. 支援範圍

Motor profile

Motor profile	狀態	控制模式
AK10-9 V3.0 KV60	可供目前硬體 backend 使用	MIT Control Mode
AK70-10 KV100	僅預留型號，控制參數尚未建立	不可用於實機

AK10-9 目前使用的命令範圍為：

項目	範圍
Position	-12.56~12.56 rad
Velocity	-33~33 rad/s
Torque	-65~65 N·m
Kp	0~500
Kd	0~5

這些範圍是通訊 codec 的數值範圍，不代表適合目前機構、供電或負載的安全操作範圍。實際操作仍以畫面中的安全限制與硬體規格為準。

Backend

Backend	用途	所需硬體
STM32（預設）	正式通訊架構	STM32、CAN transceiver、馬達
Direct CAN（Debug）	由 Python 直接操作 CAN adapter	相容 python-can 的 adapter、馬達
Simulation	無硬體測試 GUI 與基本命令流程	無

4. 安裝與啟動

環境需求

- Windows 為主要桌面執行環境。
- Python 3.10 以上。
- STM32 backend 需要可辨識的 serial/VCP port。
- Direct CAN backend 需要已安裝並設定好的 CAN adapter driver。

從原始碼執行

在專案目錄執行：

```
python -m pip install -r requirements.txt
python .
```

若電腦上有多套 Python，請確認安裝相依套件與啟動程式使用同一個 interpreter：

```
python -c "import sys; print(sys.executable)"
```

使用打包版本

執行：

```
python build.py
```

成功後產生：

```
dist/MotorControl.exe
```

5. 主畫面說明

左側有三個操作頁籤：

1. **Motor 通訊**：選擇馬達、backend、CAN 與安全設定。
2. **Feedback Check**：確認馬達回授方向與速度尺度。
3. **Manual Control**：手動送出 MIT 邏輯命令。

右側 **Motor Telemetry** 顯示即時馬達與 CAN 狀態。

當 Feedback Check 正在執行時，Manual Control 會停用；Manual Control 已連線時，Feedback Check 也會停用。兩者不會同時占用同一個 backend。

6. Motor 通訊設定

Motor

欄位	說明	建議/限制
Motor Model	馬達 profile	實機目前選 AK10-9 V3.0 KV60
Motor ID	CAN 馬達 ID	0~255，必須與馬達設定一致
Control Mode	馬達韌體控制模式	AK10-9 目前僅 MIT Control Mode

STM32 / Backend

欄位	說明
Backend	選擇 STM32、Direct CAN 或 Simulation
STM32 Port	STM32 的 USB CDC、VCP 或 UART-to-USB port
STM32 Baud	Serial baud rate；預設 115200
Connection Status	顯示 backend 測試結果

STM32 Port 欄位可以手動輸入，也可以按重新整理重新掃描 serial ports。只有選擇 STM32 backend 時會顯示 STM32 Port 與 Baud。

CAN

欄位	說明	預設值
Bitrate	STM32 CAN 或 Direct CAN bitrate	1,000,000 bit/s
Command Rate	STM32 CAN 控制頻率或 Direct CAN 命令更新率	200 Hz
CAN Interface	<code>python-can</code> interface，只供 Direct CAN 使用	<code>socketcan</code>
CAN Channel	Direct CAN channel	<code>can0</code>

Direct CAN 常見設定範例：

平台/裝置	CAN Interface	CAN Channel 範例
Linux SocketCAN	<code>socketcan</code>	<code>can0</code>
Windows PCAN	<code>pcan</code>	<code>PCAN_USBBUS1</code>
SLCAN	<code>slcan</code>	依系統裝置名稱
python-can 虛擬匯流排	<code>virtual</code>	自訂一致的 channel 名稱

Motor Safety Limits

欄位	作用	預設值
Torque Limit	拒絕超過此絕對值的手動 torque 命令	20 N·m
Current Limit	回授電流超過此絕對值時停止 worker	25 A
Temperature Limit	馬達回報溫度超過此值時停止 worker	70 °C

安全值必須大於零。第一次實機測試請依機構、驅動器、電源與馬達條件降低限制，不要直接把數值設到 codec 最大值。

測試 Motor Backend

設定完成後按 `測試 Motor Backend`：

1. 程式建立選定的 backend。
2. 開啟 serial 或 CAN 連線。
3. 最多等待約 3 秒接收 motor telemetry。
4. 成功時顯示位置、速度、溫度與 CAN TX/RX 計數。
5. 測試結束時送出 Motor Idle 並關閉 backend。

如果顯示「backend 已開啟但沒有 telemetry」，請依序確認：

- 馬達與 STM32/adapter 已供電。
- CAN_H、CAN_L、GND 與終端電阻正確。
- Motor ID、bitrate 與 STM32 韌體設定一致。
- STM32 Port/Direct CAN interface/channel 選擇正確。
- STM32 韌體已依本專案 protocol 回傳 MotorTelemetry。

7. 建議的第一次操作流程

使用 Simulation 熟悉介面

1. 在 **Motor 通訊** 選擇 **Simulation**。
2. Motor Model 選 AK10-9 V3.0 KV60，Control Mode 選 MIT。
3. 按 **測試 Motor Backend**，確認連線成功。
4. 切到 **Manual Control**，按 **Connect Backend**。
5. 選 **Velocity command through MIT**。
6. Velocity 設為 **1 rad/s**、Kd 設為 **2**，按 **Send Logical Command**。
7. 觀察右側速度、位置與 CAN TX/RX。
8. 按 **Software Stop / Motor Idle**，最後按 **Disconnect**。

第一次 STM32 實機連線

1. 斷開馬達動力或保持 Hardware E-Stop 動作。
2. 完成 STM32、CAN transceiver、馬達及共同接地接線。
3. 啟動程式並選擇 **STM32 backend**。
4. 選擇 STM32 Port，確認 baud、Motor ID 與 CAN bitrate。
5. 設定保守的 torque/current/temperature limits。
6. 解除 Hardware E-Stop 前，確認馬達已固定且活動範圍淨空。
7. 按 **測試 Motor Backend**，先確認可持續收到 telemetry。
8. 執行 **Feedback Check** 確認方向與尺度。
9. 確認無誤後才進入 **Manual Control**，從低增益與低命令開始。

8. Manual Control

連線與斷線

1. 先在 **Motor 通訊** 完成設定。
2. 進入 **Manual Control**。
3. 按 **Connect Backend**。
4. 閱讀安全確認對話框，確認 Hardware E-Stop 可用後才按 Yes。
5. 程式最多等待約 3 秒取得第一筆 motor telemetry。

連線後才可送出命令、Motor Idle 或 Disconnect。按 **Disconnect** 時，worker 會在結束前嘗試送出 Motor Idle 並關閉 backend。

Command Intent

目前 Manual Control 的四種意圖全部使用 MIT Control Mode 封包：

Command Intent	實際送出的 MIT 欄位	用途
MIT Hybrid	Position、Velocity、Kp、Kd、Torque 全部使用	混合控制
Position command through MIT	Position、Kp、Kd	以 MIT 實作位置命令，不是原生 Position Mode
Velocity command through MIT	Velocity、Kd	以 MIT 實作速度命令，不是原生 Velocity Mode

Command Intent	實際送出的 MIT 欄位	用途
Torque command through MIT	Torque，且 $K_p=0$ 、 $K_d=0$	MIT feedforward torque

按 **Send Logical Command** 只送出目前選擇的邏輯命令。STM32 backend 會將 application command 交給 STM32；Direct CAN backend 則由 Python 直接編碼 MIT CAN frame。

欄位使用方式

- Position：目標位置，單位 rad。
- Velocity：目標速度，單位 rad/s。
- K_p ：位置比例增益。
- K_d ：速度阻尼增益。
- Feedforward Torque：前饋扭矩，單位 N·m。

建議起始方式：

- Position through MIT：先用很小的角度差、低 K_p 與適量 K_d 。
- Velocity through MIT：先用低速度， K_d 可由 1~2 的保守值開始。
- Torque through MIT：先用接近零的扭矩逐步增加。
- MIT Hybrid：確認各單一意圖正常後再使用。

Software Stop / Motor Idle

按下後會將一筆全零的 Idle 命令加入 worker 的命令佇列。若 backend 或通訊已失效，這筆命令可能無法送達。因此異常時仍應立即使用 Hardware E-Stop。

自動停止條件

Manual Control 遇到下列情況會停止並嘗試 Motor Idle：

- motor error code 不為零。
- 回授電流超過 Current Limit。
- 回授溫度超過 Temperature Limit。
- 手動 torque 命令超過 Torque Limit。
- backend 開啟後約 3 秒仍無 telemetry。
- serial/CAN/backend 發生例外。

9. Motor Feedback / Scaling Check

此流程用馬達本身的命令與回授，初步確認位置方向及速度尺度。

設定

欄位	說明	可輸入範圍
測試速度	透過 MIT velocity command 送出的速度	-10~10 rad/s，不能為 0
持續時間	速度測試執行時間	1~30 秒

開始按鈕只有在以下兩項都勾選後才會啟用：

- 馬達已固定，活動範圍內沒有障礙物。
- Hardware E-Stop 已確認可立即使用。

執行步驟

1. 程式開啟 backend 並等待第一筆 telemetry。
2. 先送出 Motor Idle。
3. 顯示第一筆位置，要求操作人員安全地手動轉動輸出軸約 +90°。
4. 讀取第二筆位置並顯示位置回授變化。
5. 再次要求確認測試區域安全。
6. 以設定速度及 $K_d=2.0$ 送出 MIT velocity command。
7. 測試期間持續檢查 telemetry、motor error、current limit 與 temperature limit。
8. 結束後送出 Motor Idle。
9. 以位置變化/時間計算平均速度，並與馬達速度回授平均值交叉比較。

這項檢查只能發現明顯的方向或尺度問題，不是精密校正，也不證明馬達在負載下的扭矩或動態性能。

按 **Software Stop / Motor Idle** 可要求中止流程；worker 清理時也會再次嘗試送出 Motor Idle。

10. Motor Telemetry

右側畫面顯示：

欄位	內容
狀態	manual 、 position_check 、 speed_check 等目前階段
Backend	stm32 、 direct_can 或 simulation
馬達位置	馬達位置回授
馬達速度	rad/s
馬達電流	A
馬達溫度	°C
命令扭矩	若目前 telemetry update 有提供則顯示
CAN TX / RX	CAN 傳送與接收累計數
CAN Errors	TX error 與 RX error 的合計
MCU → CAN TX	STM32 收到命令到送出 CAN 的時間
CAN Response	STM32 送出 CAN 到收到馬達回覆的時間

下方曲線顯示電流與溫度的近期變化。若環境沒有 **pyqtgraph**，文字 telemetry 仍可使用。

11. Backend 詳細說明

STM32 backend

開啟時 Python 會先送出 **BackendConfiguration**，內容包括：

- Motor ID、motor profile、control mode。
- CAN bitrate 與 command rate。
- Torque/current/temperature safety limits。
- Host heartbeat、motor command 與 CAN feedback timeout。

之後 Python 會啟動：

- 接收執行緒：持續接收 MotorTelemetry。
- Heartbeat 執行緒：預設約 10 Hz 傳送 host heartbeat。

STM32 必須實作 watchdog：heartbeat、motor command 或 CAN feedback 超時時進入 Motor Idle。Wire format 見 [firmware/stm32/protocol.md](#)。

Direct CAN backend

此 backend 是除錯路徑，目前限制如下：

- 只驗證 AK10-9 V3.0 KV60。
- 只支援 MIT Control Mode。
- Python 必須能透過 **python-can** 開啟指定 interface/channel。
- 控制迴圈與作業系統排程不具 STM32 即時性，不建議作為正式控制架構。

Simulation backend

Simulation 會依最後一筆 velocity command 積分位置，以 torque command 值模擬電流，溫度固定為 25 °C，錯誤計數為零。它適合驗證 UI 與命令流程，不代表實體馬達動態。

12. STM32 application protocol 摘要

每個 serial frame 使用：

```
magic + version + packet type + payload length + payload + CRC32
```

主要 packet：

Packet	方向	用途
BackendConfiguration	Python → STM32	馬達、CAN、安全與 watchdog 設定
MotorCommand	Python → STM32	MIT/控制命令與 host timestamp
Heartbeat	Python → STM32	Host 存活狀態與最後命令 sequence
MotorTelemetry	STM32 → Python	馬達回授、error、CAN 統計與 timestamps

MotorTelemetry 只包含馬達/CAN 通訊路徑的資料。完整欄位、型別、unknown value 與 CRC 規則請見 [firmware/stm32/protocol.md](#)。

13. 常見問題排除

STM32 Port 清單沒有裝置

- 確認 USB 線支援資料傳輸。
- 確認 Windows Device Manager 是否出現 COM port。
- 安裝正確的 VCP/UART driver。
- 按重新整理，或直接輸入 COMx。
- 確認沒有其他程式占用該 port。

Backend opened but no motor telemetry was received

- 確認馬達供電與 CAN wiring。
- 確認 Motor ID 與 bitrate。
- 確認 STM32 firmware 已執行 CAN loop 並回傳 protocol v1 MotorTelemetry。
- 確認 Direct CAN adapter driver 與 channel。
- 檢查 CAN RX 是否持續為零、CAN Errors 是否增加。

Direct CAN 無法開啟

- 使用與作業系統/adapter 相符的 python-can interface。
- Windows PCAN 通常不是 socketcan/can0，應改用 pcan/PCAN_USB1。
- Linux 先確認 ip link 中的 CAN interface 已啟用且 bitrate 正確。
- 確認 adapter 沒有被其他程式獨占。

一送命令就停止

檢查狀態列訊息，常見原因：

- Torque Limit 設得低於命令 torque。
- 馬達回報 current 或 temperature 超過限制。
- motor error code 不為零。
- feedback 超時或通訊中斷。
- Control Mode 與 backend 命令不相符。

AK70-10 無法選擇 Control Mode

這是預期行為。目前專案沒有 AK70-10 的官方 codec 範圍、CAN frame 與控制參數，因此禁止用未驗證資料控制實機。

Software Stop 沒有反應

Software Stop 依賴相同的 serial/CAN 通訊路徑。如果通訊已失效，它可能無法送達。立即使用 Hardware E-Stop，確認馬達停止後再診斷通訊。

14. 正常關閉程式

1. 在 Manual Control 按 Software Stop / Motor Idle。
2. 按 Disconnect。
3. 確認馬達停止且 telemetry 不再更新。
4. 關閉程式。
5. 依設備安全程序切斷馬達動力。

若關閉視窗時通訊仍在執行，程式會要求確認，接著嘗試 Motor Idle 並等待背景 worker 結束。若 worker 無法停止，程式會拒絕直接關閉並顯示警告。

15. 開發者驗證

執行全部測試：

```
pytest -q
```

檢查 Python 語法與 import：

```
python -m compileall src tests __main__.py config.py build.py
```

測試 GUI 時可先選 Simulation backend，避免在介面開發階段誤動實體馬達。

16. 專案結構

```
motor_control/  
├── __main__.py          # GUI 入口  
├── config.py            # 預設 motor/backend/CAN/safety 設定  
├── ak_can.py            # AK10-9 Direct CAN debug driver  
├── build.py             # Windows one-file build  
├── requirements.txt  
├── firmware/stm32/  
│   ├── README.md       # STM32 韌體責任  
│   └── protocol.md      # Serial application protocol  
├── src/  
│   ├── communication/  # Packet codec 與 serial transport  
│   ├── controllers/    # UI 與 worker 協調  
│   ├── devices/        # STM32、Direct CAN、Simulation backends  
│   ├── models/         # Command、telemetry、timestamps、safety  
│   ├── motors/         # Motor profiles、control modes、MIT codec  
│   ├── ui/             # Qt panels 與主工作區  
│   └── workers/        # Manual Control 與 Feedback Check threads  
└── tests/              # Backend、packet、profile、UI 與 validation tests
```

開始實機操作前，請再次確認：馬達固定、旋轉範圍淨空、安全限制保守、telemetry 正常，而且 Hardware E-Stop 就在手邊。